



FILE ALLOCATION AND FREE SPACE MANAGEMENT

File Control Block (FCB)

- Stores key file information:
 - Inode number
 - Permissions
 - File size
 - Creation, modification, and access dates

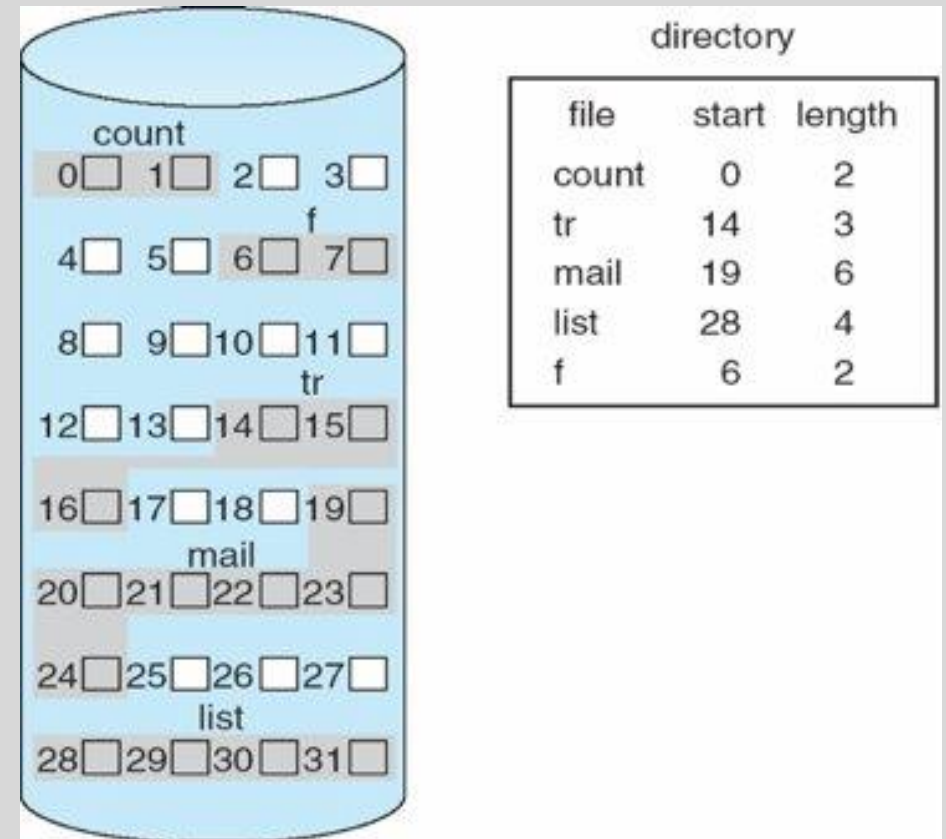
file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Allocation Method: Contiguous

- Files occupy consecutive blocks
- Best performance in most cases
- Simple: only starting block number and length needed
- Challenges:
 - Finding enough contiguous space
 - External fragmentation
 - File size must be known
 - Requires compaction (offline or online)

Contiguous Allocation

- Mapping from Logical to Physical Addresses
 - **Block to be accessed** = Starting address
 - **Displacement within block** = Offset (length)



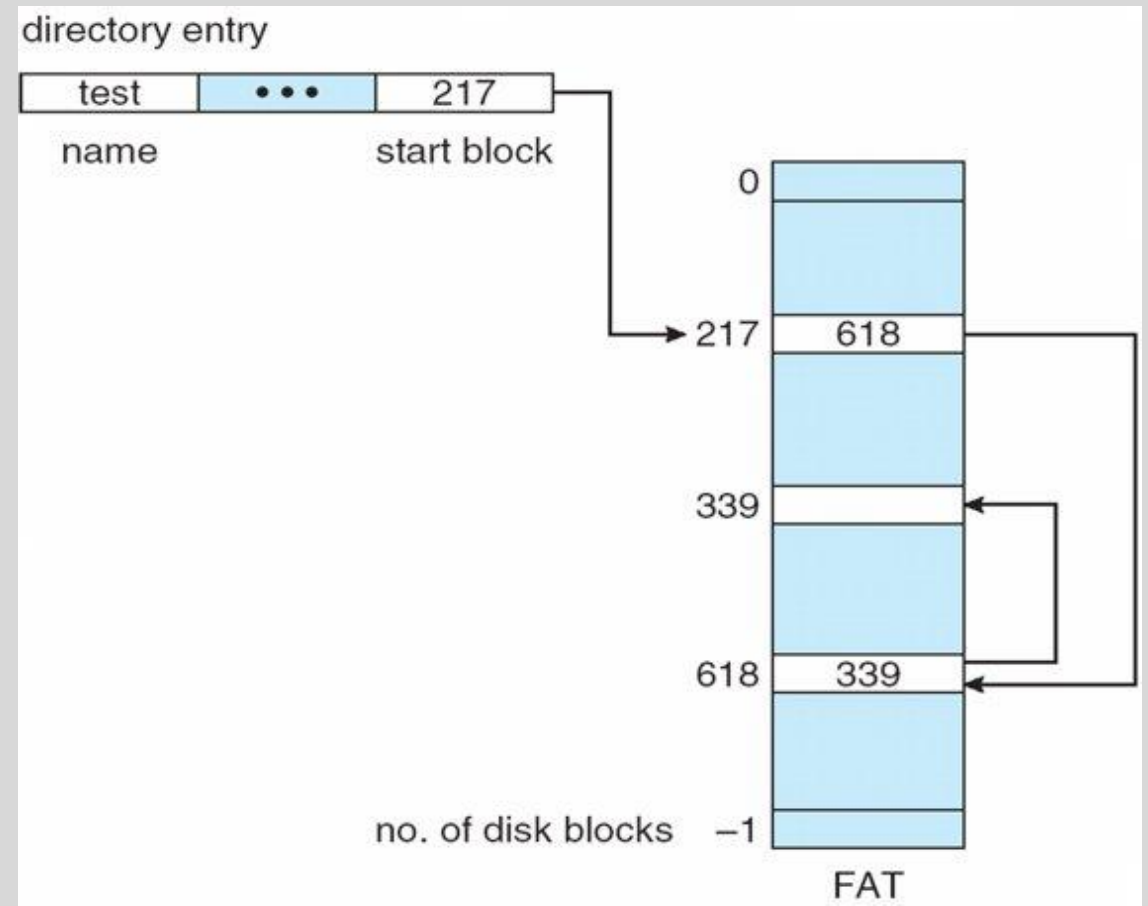
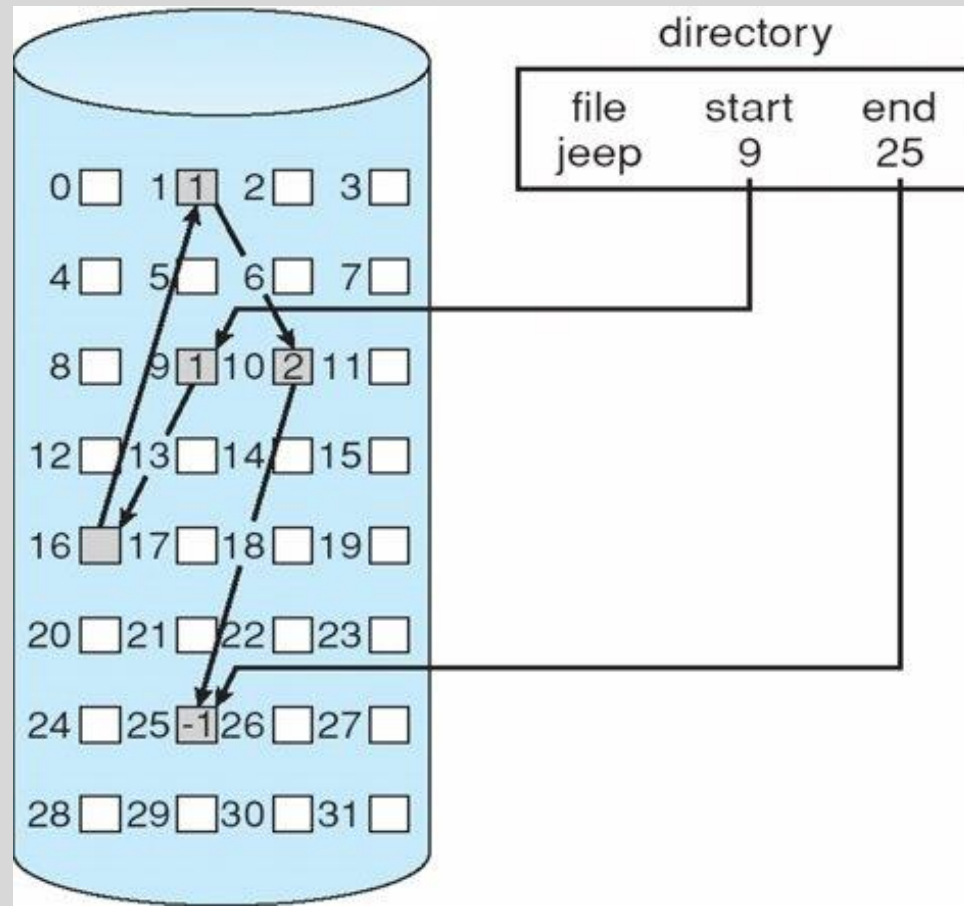
Extent-Based File Systems

- Traditional contiguous allocation assigns consecutive blocks for a file but can cause fragmentation and space issues.
- Modern file systems like the Veritas File System improve this by using **extents**.
- An **extent** is a large, continuous block of disk space allocated at once, rather than allocating single blocks.
- Files are stored as a collection of one or more extents, which reduces fragmentation and improves performance.
- This method combines the simplicity and speed of contiguous allocation with better space management.

Linked Allocation

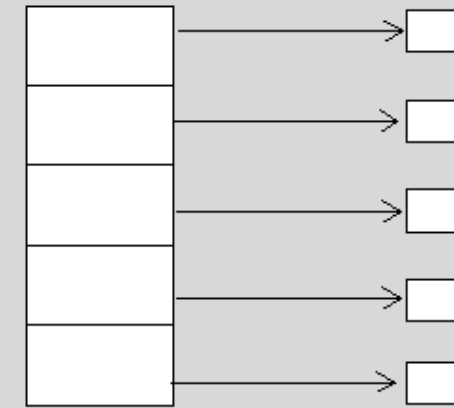
- Each file is stored as a linked list of disk blocks
- File ends with a **nil** (null) pointer
- No external fragmentation since blocks can be anywhere
- Each block contains a pointer to the next block
- No need for compaction or continuous free space
- Free space management system allocates new blocks as needed
- Efficiency improved by clustering blocks, but this can cause **internal fragmentation**
- Reliability issues may arise due to pointer corruption
- Locating a specific block may require many disk seeks and I/O operations

Linked Allocation

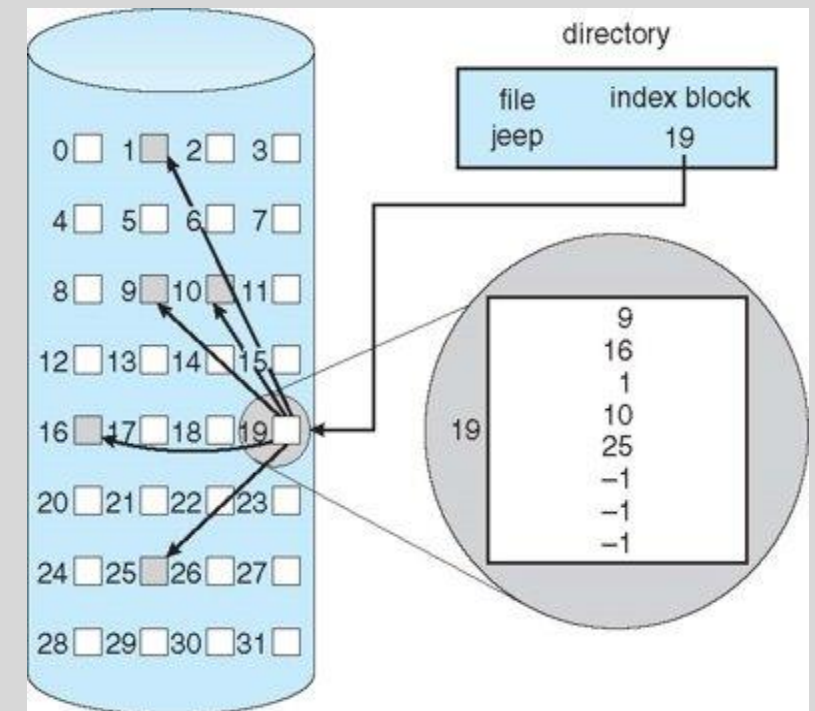


Indexed Allocation

- Each file has an **index block** containing pointers to all its data blocks
- The index block holds the addresses of the file's blocks, providing direct access to any block
- Supports **random access** to file blocks without sequential traversal
- Eliminates external fragmentation since blocks can be anywhere on disk
- The size of the index block limits the maximum file size unless multilevel indexing is used

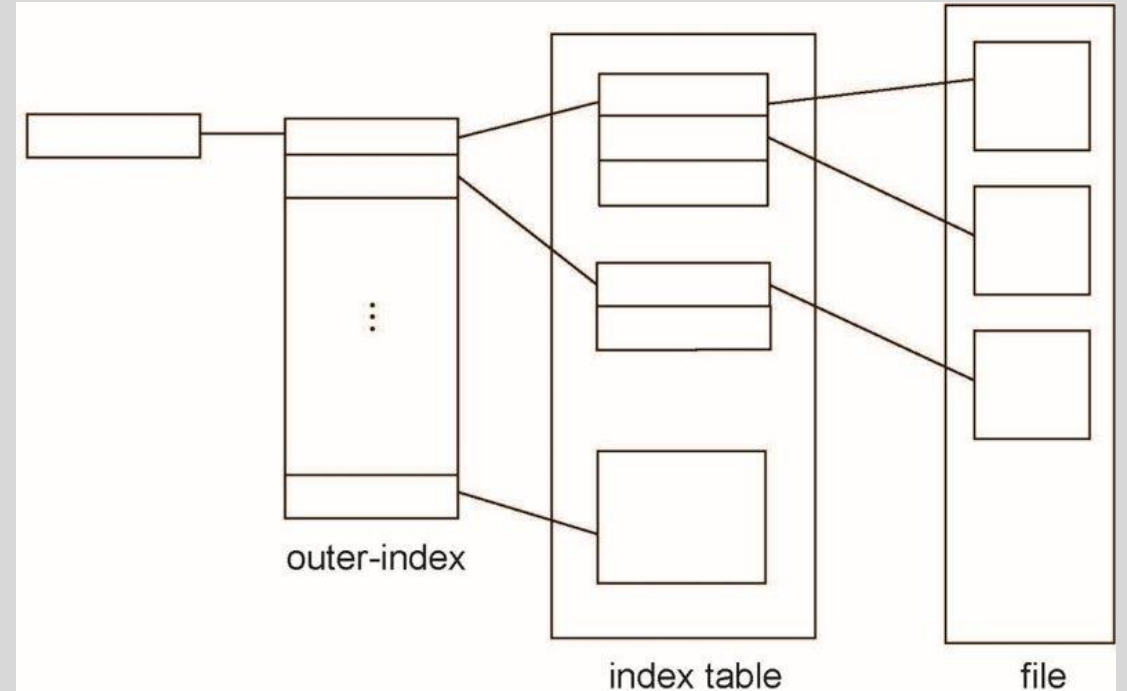


index table



Multilevel index

- Uses a **first-level index block** pointing to multiple **second-level index blocks**
- Second-level blocks point to the actual file data blocks
- OS accesses data by navigating first-level → second-level → data block
- Can extend to 3rd or 4th level for larger files



Two-Level Indexed Allocation Example

- Each index block size: 4,096 bytes
- Pointer size: 4 bytes
- Pointers per index block: $4096/4=1024$
- First-level index block points to 1,024 second-level index blocks
- Each second-level index block points to 1,024 data blocks
- Total data blocks =

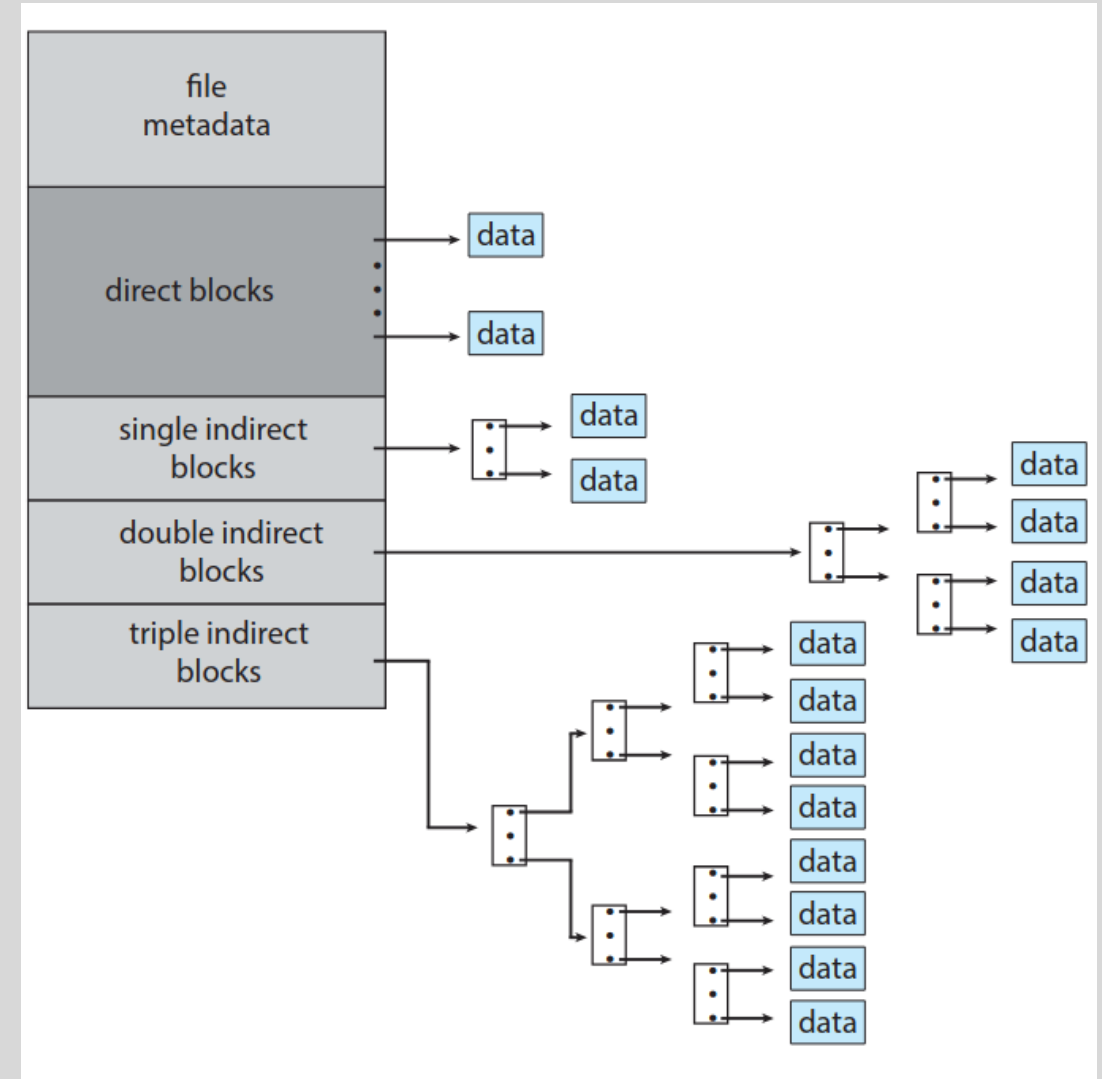
$$1024 \times 1024 = 1,048,576$$

- If each data block is 4,096 bytes, max file size =

$$1,048,576 \times 4,096 = 4 \text{ GB}$$

The UNIX inode

- In UNIX file systems, each file has an **inode** storing **15 pointers**:
- **Direct Pointers (12)**
 - Point directly to **data blocks**
 - For small files (≤ 12 blocks) $\rightarrow$ no index block needed
 - With 4 KB blocks $\rightarrow 12 \times 4 \text{ KB} = 48 \text{ KB}$ of direct access
- **Indirect Pointers (3)**
 - **Single Indirect**
 - Points to a block that holds addresses of data blocks
 - **Double Indirect**
 - Points to a block $\rightarrow$ that points to another block $\rightarrow$ that holds addresses of data blocks
 - **Triple Indirect**
 - Three-level structure:
Inode $\rightarrow$ Block $\rightarrow$ Block $\rightarrow$ Block $\rightarrow$ Data blocks
- Supports both small and very large files efficiently



Performance vs. File Access Type

- **Contiguous Allocation**

- Best for **sequential** and **random** access
- Fastest access due to block locality

- **Linked Allocation**

- Good for **sequential** access only
- Poor for random access (requires pointer traversal)

- **Indexed Allocation**

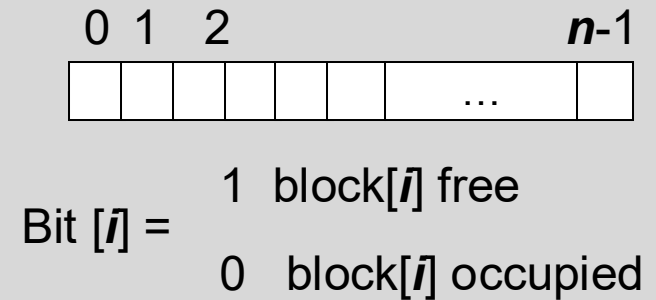
- Supports both access types
- Requires multiple reads (for index blocks)
- More complex, but flexible

- **Strategy:**

- Declare access type at file creation
- OS selects **contiguous** or **linked** accordingly

Free-Space Management

- File system keeps track of free (available) disk blocks using a **free-space list**
- Common methods to represent free space:
- **Bit Vector (Bitmap):**
 - One bit per block (0 = free, 1 = allocated)
 - Efficient for checking and allocating blocks
- Can handle large disks with simple, compact representation

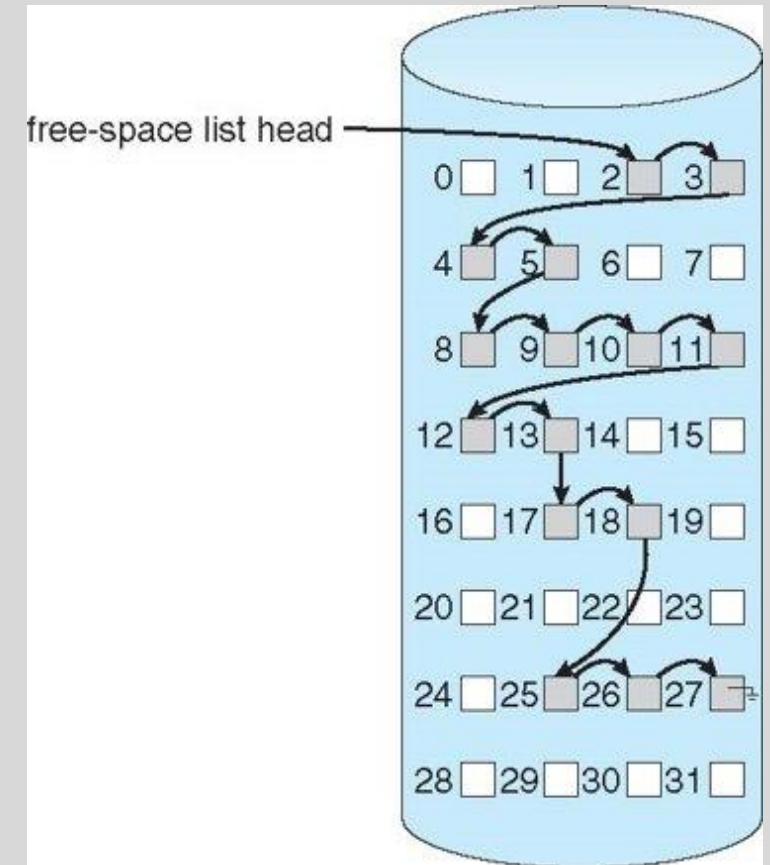


Bit Map – Space Overhead Example

- Bit map requires extra space to track free blocks
- Example:
 - **Block size** = 4 KB = 2^{12} bytes
 - **Disk size** = 1 TB = 2^{40} bytes
 - Number of blocks $n = \frac{2^{40}}{2^{12}} = 2^{28}$ blocks
 - Bit map size = 2^{28} bits = $2^{28} / 8$ bytes = **32 MB**
- Bit map is still **compact** and makes it **easy to find contiguous free blocks**

Linked List (Free List) for Free Space Management

- Free blocks linked together as a list (free list)
- **Advantage:**
 - No extra space wasted on tracking (no bitmap overhead)
 - If number of free blocks is recorded, no need to traverse entire list for allocation
- **Disadvantage:**
 - Difficult to find **contiguous free blocks**
 - Can lead to fragmentation and slower allocations



Advanced Free Space Management Techniques

◦ **Grouping**

- Enhances linked list by storing addresses of next **n-1** free blocks in the first free block
- First free block points to another block holding additional free-block pointers
- Reduces overhead of traversing many small blocks

◦ **Counting**

- Leverages contiguous free blocks often found in allocation methods like extents or clustering
- Keep address of the **first free block** and a **count** of how many contiguous free blocks follow
- Free space list entries store **(address, count)** pairs, simplifying allocation of contiguous blocks

Space Maps in Free Space Management

- **Metaslabs:** The storage device is divided into units called metaslabs.
- **Space Maps:** Each metaslab has an associated space map that tracks free and allocated blocks within it.
- **Counting Algorithm:** Space maps utilize a counting algorithm to efficiently manage contiguous free space.
- **Scalability:** This method allows the file system to scale effectively, managing large volumes of data with improved performance.

Factors Influencing File-System Efficiency

- **Disk Allocation and Directory Algorithms**

The choice of how disk space and directories are organized impacts access speed and storage overhead.

- **Information Stored in Directory Entries**

More metadata (like timestamps, permissions) increases overhead but supports richer features.

- **Pre-allocation vs. On-demand Metadata Allocation**

Pre-allocating space for metadata can improve performance but may waste space; on-demand saves space but might cause delays.

- **Fixed-size vs. Variable-size Data Structures**

Fixed-size structures simplify management but can cause internal fragmentation; variable sizes optimize space but increase complexity.

Performance Considerations

- **Data & Metadata Proximity**

Storing metadata close to data speeds up access.

- **Buffer Cache**

Memory area holding frequently used blocks to reduce disk reads/writes.

- **Synchronous Writes**

Data written directly to disk immediately for reliability, but slower.

- **No Buffering**

Writes bypass cache, guaranteeing save but with higher latency.

- **Asynchronous Writes (Common)**

Writes buffered in memory and saved later, improving speed but risking data loss on crash.

Page Cache and Buffer Cache

- **Page Cache**

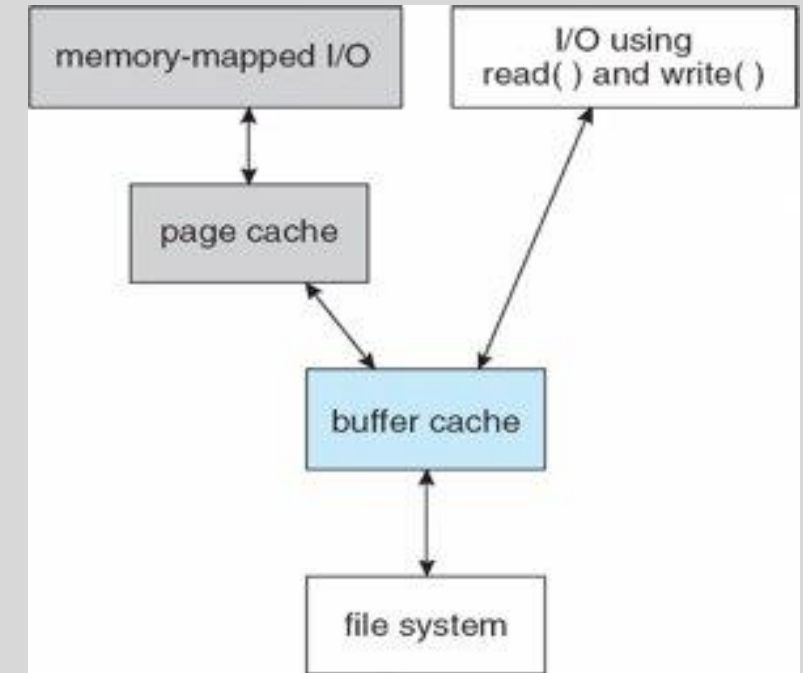
Caches entire memory pages (not just disk blocks) using virtual memory mechanisms. This allows efficient access by mapping files directly into a process's address space.

- **Memory-Mapped I/O**

Uses the page cache to access file data via memory addresses, enabling faster and simpler file operations.

- **Buffer (Disk) Cache**

Used for routine file system I/O, caching smaller disk blocks to optimize read/write operations without memory mapping.

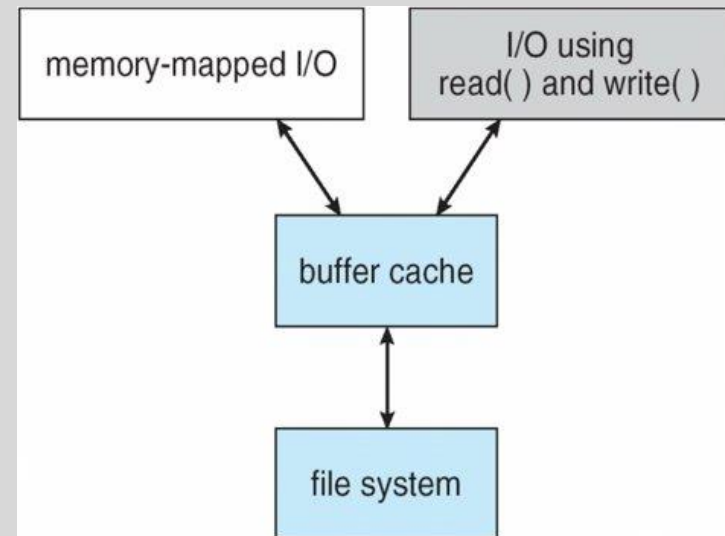


Unified Buffer Cache

- **Concept:** Uses a single page cache for both
 - **Memory-mapped pages**
 - **Ordinary file system I/O**
- **Why?**

Avoids **double caching** of the same data in separate caches (page cache + buffer cache).
- **Benefit**

Improves memory efficiency and consistency by maintaining one shared cache for all file data.



Consistency Checking and Backup

- **Consistency Checking**

Compares directory metadata with actual data blocks on disk to find and fix errors.

- Can be slow
- May sometimes fail to resolve issues

- **Backup**

Use system tools to copy data to other storage (tape, disk, optical media) for safekeeping.

- **Recovery**

Restore lost files or damaged disks by retrieving data from backups.

Log-Structured (Journaling) File Systems

- **Metadata Updates as Transactions**

Every change to the file system's metadata is recorded as a transaction.

- **Transaction Log**

All transactions are first written sequentially to a log.

- **Commitment**

A transaction is considered complete (committed) once it's safely written to the log.

- **Asynchronous Update**

Transactions are later applied to the actual file system structures asynchronously.

- **Crash Recovery**

After a crash, any uncompleted transactions in the log are replayed to ensure consistency.

- **Benefits**

Faster recovery and eliminates metadata inconsistencies caused by crashes.



Thank You